General

Marker

- Test123
- Test123
- Test123
- Test123
- Test123

Abstract

Marker

TODO

Summary

Marker

TODO

Notes

```
<@ actual = expected @>
<@ actual @> = expected

<@ actual @> < expected
<@ actual @> ⩽ expected
<@ actual @> > expected
<@ actual @> ⩾ expected

<@ actual @> throws
<@ actual @> throwsAny
<@ actual @> isTrue
<@ actual @> isFalse

<@ actual @> Expectation?  [refValue]
```

```
<@ fun tuppledArgs -> actual tupledArgs = expected tupledArgs @>
<@ fun tuppledArgs -> actual tupledArgs @> = expected tupledArgs

<@ fun tuppledArgs -> actual tupledArgs @>
=
fun tuppledArgs -> expected tupledArgs

<@ fun tuppledArgs -> actual tupledArgs @>
Expectation?
fun tupledArgs -> expected tupledArgs
```

1 Introduction

Motivation

I have been in contact with a lot of programming languages: C++, Python, Java, F# and they all have similar error messages, since the "concept" is similar "print exception plus stacktrace". But then again a lot of details matter when working with error messages or code in general. The definition of a "novice" in my context is something like: A person that has not had much contact with programming on their own In my very specific case these persons are often first or second semester students on a university (or possibly higher) but they all have not yet passed the exam of a introductory course for programming. So even if they are higher semester students there are probably a few that have not been able to master certain skills yet, besides neglectable exceptions for not passing that exam like disabilities, personal struggles, etc.

I have been a tutor for almost as long as I am a university student myself and I have noticed over and over again that novices are not able to interact with code like we try to teach them. Of course a big part is teaching, although I would argue that conventions, principles and other stuff are pretty well explained and taught at most universities and that is not my topic. At the end of the day most students that try to learn how to program have to sit behind their PC and program for themselves. And I notice that when that happens they are often overwhelmed with the combination of code, IDE, output, etc.

I want to look specifically at the error messages. Because as Barik or the other paper show, programmer spend a lot of time with error messages. Novices even more so.

Specifically I want to look at the relation between the toal information displayed and the actual crucial information given in that amout of data. Especially look or discuss the topic of verosity a bit, that one might incrementally reveal information to students on their learning journey, so a framework with such methods would be cool to write tests like normal and offer them to various students at various levels of verbosity without having to change a lot.

The most crucial information is not at what bit at which address in the memory did your while loop fail? and a stacktrace of 1000 internal files and methods that most people wouldnt even know how to open nor read. For a novice that is supposed to write leastCommonDenominator, add, mul, mod5 or fibonacci(n), etc. the most crucial information is often. (Fib as example) fib(0) was supposed to be 0, not 1. or better formatted: """ - Tested was: fib 0 - Expected was: 0 - Given was: 1 => Your implementation is wrong and gives the wrong output for value 0 """

I think it is worth looking into a framework that is constructed like that. Not only in teaching but self-learning as well. If implemented correclty, so that users are able to EASILY write running code or test their not-running code and receive various levels of error message detail. And to learn to appreciate the certain features or error messages along the way as concepts get more complex and the needed information from tests gets more complex as well.

So to set a realistic goal i wanna find out what would good tests look like? Can i derive implementable Requirements from that and realize them? How useful is the result? In the end I wanna merely discuss the bigger picture: What would be possible and how likely is it that people would benefit from that. I am imagining of a self-reporting-learning-framework. A framework that offers not only methods at various levels of verbosity like print, printfn, System.Console.Write, etc but also a testing library that is easy to use and give result at various levels of verbosity as well. Then not only the typical error message is printed but it might be useful to considering some kind of "reflection" on the implemented code by the framework itsself. The results would only be good the less you know. Because you make the same mistakes over and over again. I did some analysis on submissions and noticed a exponential curve of error messages. The top 10 of error messages covers like 90% of all error messages. So the system could learn that in very secific cases the error might be so obvious that the framework can even give

a more detailed explanation as to why that error happened, how to fix and to avoid it. Maybe even describe similar scenarios.

I formulated these research questions: RQ1 (Problem characterization): Which structural characteristics of compiler and testing-framework outputs make failing tests difficult to interpret for novice programmers in introductory programming exercises?

RQ2 (Design requirements): Which requirements can be derived for novice- oriented test feedback with respect to consistency, cognitive load, and the action- ability of failure information?

RQ3 (Reporting scheme): How can a uniform tested expression/expected/ac- tual reporting scheme be defined such that it remains applicable across different kinds of tests (example-based unit tests and property-based tests)?

RQ4 (Technical feasibility): How can this reporting scheme be implemented in F# using Quotations and Unquote for expression capture and evaluation, while remaining compatible with existing tooling through wrappers around Microsoft.VisualStudio.TestTools.UnitTesting and FsCheck?

RQ5 (Results and discussion): To what extent does the implemented artifact (Assertify & Checkify) satisfy the derived requirements in practice, and what limitations and trade-offs emerge from the wrapper-based approach with respect to expression coverage, supported assertion forms, formatting constraints, and integration boundaries?

1.1 Problem Statement and Objectives

Marker

see motivation for chapter

1.2 Research Questions

Marker

see motivation for chapter

1.3 Thesis Structure

Marker

see motivation for chapter

2 Background and Related Work

Motivation

Marker

sections, other concrete literature besides barik, ..

Here I wanna show with literature that error messages are worth looking into. As barik already found out: Novices/Programmers spend a lot of time reading error messages. Here are some questions i have in mind when looking for new resources/papers: - How could one quantify the "goodness/badness" or error messages and are there attempts to that? - What role do error messages play in learning (learning to code)? - Are there attempts to improve error messages? - Are there not just attempts but already frameworks/programming languages that have achieved this to some degree? Rust? - Are there scientific requirements or definitions as to improve error

3 Problem Analysis

Motivation

Why are error message so bad? One thing might be the overwhelming amount of text in relation to the needed information (for a novice). (Informal) Some frameworks do not give a fuck and just print terrible information. There is no consistency whatsoever. Description of the course (from the module handbook): """ Students learn the basic concepts of programming languages and acquire the skills to develop programs and to learn programming languages on their own. After successfully completing the module, students will be able to describe basic concepts of programming and software design, explain the characteristics of different programming paradigms, model, implement and test small to medium-sized programs using a programming language, use advanced functional, imperative and object-oriented programming concepts and techniques, implement fundamental algorithms and data types and use them to solve problems. """ This description should give a brief mirror of the average level of skill students have that attend this course. The programming language used in this course if F#. Typically students get a template consisting of project with already predefined helper methods and types. In this project there are then for each exercise sheet further type definitions and methods with placeholders for the students implementation. Example: let add a b = failwith "TODO" As other courses do as well, this course uses "ExClaim" for the submission of students' code. Students are able to submit their code and receive the test results there. The tests are hidden from the students to 'force' them to use the website to test their code. By that data can be extracted/derived like the number of attempts before 100% successful tests, average percentage of failed tests, most common error codes/messages, the difference between the first and last submission and various other measurements that might give insight into the improvement of learning/teaching processes. In this environment A-B-Tests could compare the two approaches: "Old/Standard" vs "Improved" Do the error messages and website feedback influence any of those variables?

After the submission deadline they will eventually find a commented and 'corrected' version of their code - graded by their tutor, that combined the submitted code, test resuls and assignment to assess whether or not the task was successfully implemented. Other than the fact that it is in their browser, students receive exaclty the same result as they would in their own IDE. The tests are usually implemented using Microsoft.VisualStudio.TestTools.UnitTesting, Unquote and FsCheck Used methods are mostly AreEqual from UnitTesting, test <@ ... @> from Unquote and Check.One from FsCheck

One thing I notice is that all error messages often try to express the same thing in a different shape: "This was tested" "I wish this behavior" and "This was your (varying) behavior" Novices are often not interested in the thousand of files that are included in the failure. In >95% of cases students have to edit and submit only a single file. So the number of files that responsible reduces quite drastically (joking: there is usually just one option or the test only refers to one specific file the student edited and not multiple) That makes the stacktrace mostly if not completely useless. Then sometimes tests are called testIfArrayIsEmptyIfTenDifferenElementsAreAddedAndThenRemovedAtRandomOrder, test123 or with F#: `"Hey this test is supposed to test something"` None of that is sometimes expressive enough to reflect what was actually tested: "add of 1 and 2 should be 3" this suggests some sort of expression capture to reflect what was tested, but not revealing any sample solutions of methods. so students should not see: "add of 1 and 2 should be '1 + 2'" this would reveal the implementation "add a b = a + b" while "add of 1 and 2 should be 3" does not do that strictly speaking. So expression capture is important and to be useful a consistent format would be useful like: f(x) =! 0 (from school math to see whether a curve has a high- or low-point) => actual =! expected => add 1 2 =! 3 What other specifications can be derived that are useful for tests? Actual and Expected value is important so these three key thing would be something reasonable to implement. The goal is to extract all key

features of error messages and error reflection to explicitly derive implementation goals from that.

I got this (work-in-progress) motivations: ▪ Expression capture: obtain a representation of the tested expression that can be displayed to the user (and simplified where appropriate). ▪ Value extraction: evaluate expressions in a controlled way to compute actual values and to render values consistently. ▪ Wrapper-based integration: intercept failures from existing unit testing and property-based testing frameworks and replace or augment their output without requiring changes to the surrounding tooling. ▪ Unified output model: define a single report representation that can be produced for different assertion forms and test paradigms.

3.1 Context and Assumptions

Marker

see motivation for chapter

3.2 Observed Issues in Existing Compiler/Test Output

Marker

see motivation for chapter

3.3 Derived Requirements and Design Implications

Marker

see motivation for chapter

4 Assertify (and Checkify): Design and Implementation

Motivation

This ties directly to the previous chapter and grabs up the design requirements for tests and error reflection. From that the specific design goals are formulated. Also the conventions and principles should be explained before starting implementation. How can tests be output and besides "red text" or "blue text" things like "summary on top and more detail below" or "all in one flowing texts, no paragraph no nothing" After that some background is necessary. The topic of expression capture can be easily achieved in fsharp via quotations. These are basically just an Tree/AST/Expression represenation of actual code. Together with Unquote one can even evaluate, reduce, substitute and analyze code. With that feature the expression capture is basically done. It is just important to have a detailed documentation of quotations and how they work in the context of the implementation. But it should only be examplatory. Quotations change the interaction with the test framework so a big key challenge is value extraction. Especially for random tests it is important to not loose the input variables. Althoug this is also covered by quotations+unquote because eval, patterns and substitution helps evaluation expressions and extracting information. For FsCheck i used a implementation of the runner interface to catch the generated input values to the failed test case.

With Unquote/Quotations it is easy: actualExpr = expectedExpr => actualExpr.Eval() = actualValue => expectedExpr.Eval() = expectedValue Then the explicit implementation, algorithmic and design details are important of Assertify and Checkify. How are Quotations used and what do they achieve? What is the final result and how do outputs look like? assertify returns json strings How are these outputs then used in the test pipeline and how does the final output look like.

4.1 Design Goals and Principles

Marker

see motivation for chapter

4.2 Output Model: Representing Tests

Marker

see motivation for chapter

4.3 Mechanisms: Expression Capture and Value Extraction

Marker

see motivation for chapter

4.4 Reporting: Formatting and Integration with the Test Pipeline

5 Results and Discussion

Motivation

I designed an evaluation, but had a critical implementation error that rendered all data useless. Therefore I need to focus on what DOES work and focus less on the evaluation/measurement part.

RQ1 (Problem characterization): I analyzed and cover the problem in "Problem Analysis"

RQ2 (Design requirements): In "Problem Analysis" quantitative/qualitative(?) requirements are formulated adn stated.

RQ3 (Reporting scheme): The reporting scheme is also analyzed and requirements are stated in "Problem Analysis"

RQ4 (Technical feasibility): Here I need to be critical as to what degree Assertify/Checkify does solve the problem in context of the limitations and whether a "100% bullet proof solution" for this problem is even possible under technical aspect like compiler, GPU, runtime reflection, etc.

RQ5 (Results and discussion): Does this discussion have some scientific value to it?

What are the concrete results from Assertify and what is the "status quo" of my development? Assertify does provide two/three solid methods for testing that are able to process complex tests for novice pruposes and display the results in a manner so that visualizaion is easy (JSON format can be parsed and the information can be used however one wants) How can the results still be scientifically interpreted without evaluation? This is the question I am not sure how to answer currently. The limitations are real life scenarios that break the pattern of "actual = expected". in the current state other assertions like isTrue, isLargerThanOrEqual, throws, etc. although checkify.checktest is able to process complex tests using generators and still displays them in the same format.

5.1 Research Question Coverage and Requirement Satisfaction

Marker

see motivation for chapter

5.2 Artifact Outcomes

Marker

see motivation for chapter

5.3 Key Findings and Interpretations

Marker

see motivation for chapter

5.4 Strengths and Limitations

Marker

see motivation for chapter

6 Conclusion

Motivation

Summarize results and go about possibilities for future work: ▪ Useful Evaluation/Data once the code is running smoothly on ExClaim ▪ FSharp.Compiler.Services to reflect on what went wrong ▪ A combination of FSharp.Compiler.Services and FParsec would be able to produce better errors for majority of failures ▪ TODO DATA OF TOP 10 ERROR CODES !!!!! #1 is FS0001 expected int but got string which is a easy error to parse for. fsharp reflection tools are able to extract the problematic code together with the error message and then i would be easy to use pattern matching to notice that a int was expected but somehow chars are inside the objectString. ▪ Generalization via MiniLib: ▪ Including student friendly helper modules/types/etc ▪ Test Library with configurable verbosity to suit multiple levels of learning progress ▪ Maybe implement it as a full Test Framework ▪ Combine with VS Code and code-server? This would enable students to have a IDE experience but deliver diagnostic information to teachers/educators.

Finally what's left to say about the topic and about the thesis in general?

6.1 Summary

Marker

see motivation for chapter

6.2 Future Work

Marker

see motivation for chapter

6.3 Final Remarks

Marker

see motivation for chapter